MarketMind AI - Milestone 2 Development Report

Customer intelligence, forecasting, secured APIs and role-specific dashboard integration | 13 August 2026

FUNCTIONAL SCOPE	CURRENT EVIDENCE	EVALUATION POSITION
Customer segmentation, behaviour analysis, business revenue, personal sales, store-product demand and model monitoring.	17 backend tests, 10 preprocessing tests, Ruff, Alembic, frontend lint and production build pass.	Milestone 2 functionality is integrated for local evaluation. Production deployment is intentionally the final stage.

What Milestone 2 adds

Customer intelligence

RFM and purchasing-behaviour features, engagement scoring and three business-readable customer segments.

Forecasting engine

7/14/30-day business revenue, Sales Executive personal sales and store-product demand forecasts.

Secured integration

JWT sessions, MFA for Administrators, RBAC and tenant/store/seller-scoped FastAPI responses.

Connected dashboards

Real KPIs, charts, filters, search, tables, model metrics, exports and loading/error states.

Core outcome

End-to-end result

Cleaned source data is transformed into versioned ML outputs, imported idempotently into the relational database, exposed through role-protected APIs and displayed in four React workspaces. The frontend never reads the database directly.

1. Requirement Analysis and Objectives

The business need is translated into measurable user outcomes, security rules and technical deliverables.

Business requirement	Milestone 2 response	Acceptance evidence
Understand customer behaviour	RFM, basket, tenure, product variety, returns and transaction-engagement features	Segment summary, customer membership, search and filters
Predict future revenue	Daily net INR forecasts with 7/14/30-day horizons and prediction intervals	Business Owner revenue KPIs, chart, comparison and export
Plan store inventory	Store-product demand forecast linked to reviewed inventory mappings	Store Manager product table, trend and stock-risk indicators
Support individual performance	Personal forecast trained from authorised processed sales	Sales Executive sees only the authenticated seller scope
Monitor AI operations	Model versions, algorithms, metrics and import job status	Administrator monitoring view after MFA
Preserve access control	Backend session, permission, MFA and data-scope enforcement	401/403 behaviour and automated RBAC tests

Role objectives

Role	Objective	Permitted Milestone 2 view	Primary restriction
Business Owner	Understand business growth and customer value	Business segmentation + revenue forecast	Cannot configure AI models
Store Manager	Plan stock for the assigned store	Store summary + product demand	Cannot access business revenue or customer membership
Sales Executive	Track assigned customers and own future sales	Assigned customers + personal forecast	Cannot access another seller or store/business forecasts
Administrator	Monitor the tenant and system operation	All report types + model monitoring	MFA required; selectors remain tenant-bound

Milestone objective

Deliver usable AI-assisted analytics without weakening Milestone 1 authentication, RBAC, data quality or database integrity. Model outputs must be explainable, versioned, testable and connected to real UI states.

2. Datasets: Selection, Purpose and Business Value

Each source is assigned to a specific task; datasets are not joined unless a stable key or reviewed mapping exists.

Dataset	Why selected	Used for	How it helps
Online Retail II	Two-year invoice/customer/product history with quantity, price and returns	Customer segmentation and purchasing behaviour	Supports RFM, engagement, basket, variety and return features for 5,878 customers
Amazon sales	Dated order records with INR Amount, category and status	Business revenue forecasting	Creates daily net revenue after cancellations and return-like statuses are handled
Processed sales	Validated MarketMind sales/returns with seller assignment and INR amount	Sales Executive personal forecast	Provides a small but correctly scoped seller-scale time series
Train/eval Parquet	Large dated store-product history with business and context fields	Product demand forecasting	Supports lag, calendar, discount, activity, stock and weather relationships
Inventory data + mapping CSV	Current stock, reorder level, SKU/store references	Demand-to-stock interpretation	Adds stock risk only when the store/product mapping is explicitly valid

Data preparation and safeguards

Step	Rule	Reason
Clean	Standardise dates, identifiers, currency and numeric fields; reject invalid rows	Prevents unreliable model inputs
Engineer	Build RFM/behaviour features and lag/calendar/context forecasting features	Converts transactions into model-ready information
Evaluate	Use chronological holdout data for forecasts; compare 2-8 clusters for segmentation	Avoids random leakage and unsupported model selection
Version	Write prediction CSV, report JSON and model artifact	Makes results repeatable and traceable
Import	Use stable model/scope keys and upsert behaviour	Repeating an import does not create duplicates

Known data limits

The Amazon history is short, so revenue accuracy is prototype-level. Parquet sale_amount remains labelled source_unit until its provider confirms whether it means quantity, value or an index. Unmapped demand products do not receive invented stock risk.

3. Functionality and Module-wise Development

Milestone 2 was implemented as connected modules rather than isolated model notebooks or static screens.

Module	Developed functionality	Main UI/API output	Validation
Customer intelligence	Feature engineering, engagement score, model comparison, segment profiles and assignments	Summary KPIs, segment distribution, customer table and detail	Search, pagination, role-scoped membership
Business revenue	Category-aware daily forecast, 7/14/30 horizons, intervals and candidate metrics	Owner chart, predicted total, growth direction, comparison and CSV	Allowed horizon/category; business role
Personal sales	Separate forecast artifact from valid INR seller-scale history	Executive personal KPI, chart and table	Authenticated seller ID overrides user input
Product demand	Store-product series, model comparison, inventory mapping and stock-risk context	Manager demand table, trends, filters and risk indicators	Assigned store; reviewed SKU mapping
Model monitoring	Model versions, type, algorithm, metrics, timestamps and job counts	Administrator monitoring report	Administrator + MFA only
Authentication/RBAC	Access/refresh sessions, token rotation, MFA and permission dependencies	Role-specific modules/navigation	401, 403, inactive/expired session handling
Import pipeline	Dataset, segmentation and forecast import commands with stable keys	Database-backed dashboards	Required columns, scope IDs and duplicate-safe upsert

Frontend responsibilities

Render role workspaces; request real APIs; maintain token state; show filters, loading, empty and error states; export permitted data.

Backend responsibilities

Validate contracts; enforce sessions, permissions and scopes; query SQLAlchemy services; return predictable Pydantic responses.

ML/pipeline responsibilities

Clean and engineer features; compare models; save versioned metrics/predictions; import results without duplication.

Functional result

All four dashboards use the same secured backend and database, but receive different fields and scopes according to role. This provides one integrated product while preserving least-privilege access.

4. Code Structure and Responsibility Map

The repository separates interface, business logic, persistence, model processing, tests and documentation so each layer can change independently.

Repository tree

```
Team_1_Small_Biz_Sales_AI/
|-- backend/
| |-- app/api/v1/ # HTTP routes: auth, dashboard, segments, forecasts...
| |-- app/core/ # configuration, JWT/security, permission catalogue
| |-- app/models/ # SQLAlchemy identity, sales, customer and ML tables
| |-- app/schemas/ # Pydantic request/response contracts
| |-- app/services/ # scoped queries, imports and business logic
| |-- app/commands/ # seed, data import, segment import, forecast import
| |-- alembic/ # versioned database migrations
| `-- tests/ # authentication, RBAC, imports and API tests
|-- frontend/
| |-- src/api/ # shared HTTP client and bearer-token handling
| |-- src/context/ # authentication, data, theme and toast state
| |-- src/components/dashboards/ # four role workspaces
| |-- src/components/modules/ # customers, reports, sales, inventory
| `-- src/components/common|ui/ # navigation, filters and reusable UI
|-- preprocessing/ # cleaning, segmentation, forecasting and ML tests
|-- data/ # reviewed raw/processed samples and model outputs
|-- docs/milestone-2/ # workflow, segmentation and status documents
`-- output/pdf/ # presentation and evaluation documents
```

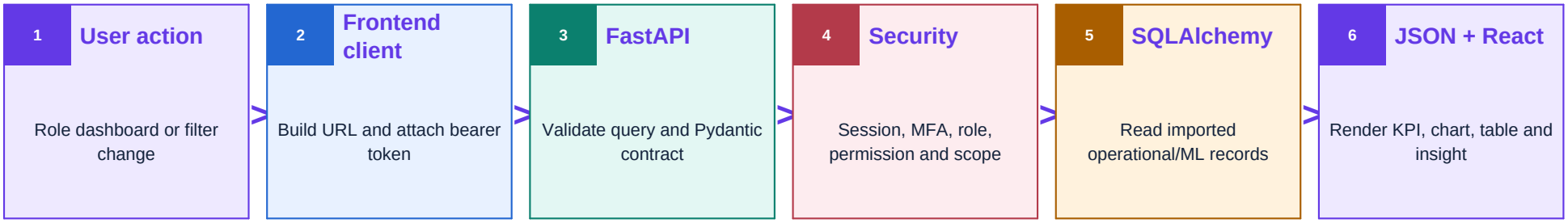
Layer responsibilities

Layer	Key files	Why this separation matters
API	api/v1/forecasting.py, segmentation.py	Routes remain thin; dependencies validate access before services run
Schemas	schemas/forecasting.py, segmentation.py	Frontend contracts are typed and automatically documented
Services	services/forecasting.py, segmentation.py	Database query and scoping logic can be tested outside the UI
Models	models/forecasting.py, segmentation.py	Operational records and versioned ML outputs have explicit relationships
Imports	commands/import_forecasts.py, import_segments.py	Offline ML generation is decoupled from online API requests
Frontend API	api/client.js, context/AuthContext.jsx	One client attaches tokens, refreshes once and normalises errors
Reports UI	modules/ReportsModule.jsx, CustomersModule.jsx	Presentation logic consumes stable API responses

12	9	10	12	10
API files	model files	schema files	service files	backend test files

5. API Integration and End-to-end Workflow

React communicates only with FastAPI; the backend performs the final security and data-scope decision.



Endpoint	Purpose	Authorised scope	Frontend use
POST /api/v1/auth/login	Issue access + refresh tokens	All roles; Admin MFA	Start authenticated session
GET /dashboard/access	Return allowed modules/actions	Current user	Build role workspace
GET /customer-segments/summary	Behaviour and segment KPIs	Business/store/assigned summary	Customer intelligence cards
GET /customer-segments	Search/filter customer membership	Owner/Admin or assigned Executive	Customer table
GET /forecasts/revenue	Business revenue series and metrics	Owner/Admin	Revenue report
GET /forecasts/personal	Seller-scale revenue series	Signed-in Executive/Admin-selected seller	Personal report
GET /forecasts/demand	Product demand, mapping and risk	Assigned Manager/Admin-selected store	Demand report
GET /forecasts/monitoring	Models, health and jobs	Administrator + MFA	Monitoring report

Token lifecycle

The frontend stores the token pair in local or session storage. It attaches access_token to protected requests. On one 401, it rotates refresh_token and retries. Logout revokes the database session. Backend checks remain authoritative.

6. Database Design and Machine-learning Implementation

Versioned model outputs live beside operational data, but remain traceable to scope, algorithm and training run.

Database area	Core tables	Purpose
Identity/security	tenants, stores, users, roles, permissions, auth_sessions, audit_events	Tenant isolation, RBAC, MFA-backed sessions and traceability
Operations	sales_transactions, products, inventory, customers	Dashboard source data and business/store/seller scoping
Segmentation	segmentation_model_runs, customer_segment_assignments	Preserve model version, metrics, profiles and per-customer membership
Forecasting	forecast_model_runs, forecast_predictions, forecast_jobs	Preserve business/store/personal scope, candidate metrics, intervals and import status

Selected models

Feature	Selected model	Why selected	Current evidence / benefit
Customer segmentation	K-Means; Hierarchical comparator	No target labels; scalable, repeatable groups after RobustScaler	5,878 customers; 3 segments; Silhouette 0.364
Business revenue	Weekday-aware Linear Trend	Lowest MAE on short Amazon validation among five candidates	MAE 77,902; ~25% below earlier XGBoost; prototype accuracy
Personal sales	Seasonal Naive	Small seller-scale history; recent weekly pattern wins MAE-first rule	207 source rows; MAE 748; seller-scoped
Product demand	XGBoost Regressor	Learns non-linear lag, date, discount, stock/activity/weather effects	250 series; MAE 2.152; R2 0.918

Model selection

Forecasts use chronological validation, Seasonal Naive baseline and candidate comparison. Selection minimises MAE, then RMSE. Segmentation evaluates 2-8 clusters using Silhouette, Davies-Bouldin and Calinski-Harabasz.

Database integrity

Alembic migrations version schema changes. Import commands validate required fields and scope IDs, then upsert stable model/scope keys so reruns update rather than duplicate records.

7. Testing, Validation, Demonstration and Deployment

Functional evidence is recorded separately from future production-hardening work.

Area	Current result	Coverage
Backend	17 tests passed + Ruff passed	Auth/MFA, RBAC, dashboard scope, customer/segment/forecast APIs, imports and Admin workflows
Preprocessing/ML	10 tests passed	Cleaning, feature generation, cluster assignment, forecast candidates, horizons and artifacts
Database	Alembic check: no new upgrade operations	Models and migrations remain aligned
Frontend	Oxlint + Vite production build passed	Compiled 2,385 modules; only non-blocking bundle-size warning

Issue / validation	Resolution or expected behaviour
Expired access token	Frontend refreshes once and retries; invalid refresh returns to login
Wrong role or scope	Backend returns 403; query cannot override assigned store/seller
Invalid horizon/filter	Pydantic/query validation returns 422; horizons limited to 7/14/30
Missing model result	API returns 404 and UI shows an empty/error state
Duplicate import	Stable upsert keys update existing run/prediction instead of duplicating
Unknown demand-inventory mapping	No false SKU link; mapping remains unmapped and stock risk unknown

Run locally

Backend: `.venv\Scripts\python.exe -m uvicorn app.main:app --port 8001`
Frontend: `npm.cmd run dev`
Open UI: 127.0.0.1:5173
Swagger: 127.0.0.1:8001/api/v1/docs

Demonstration order

1. Login as a role.
2. Open its customer/forecast report.
3. Change horizon/filter.
4. Show real API response in Swagger.
5. Switch role and demonstrate restriction.
6. Show Admin monitoring after MFA.

Deployment - final stage

Local SQLite is used for development. Docker Compose defines FastAPI + PostgreSQL for deployment. Before production: secrets, HTTPS, backups, monitoring, CI/CD, longer revenue history, confirmed demand units and load/security testing.

Evaluation: functionally integrated locally; production is not yet claimed.